Lecture 4:

Project Creation
&
CMSIS

Cortex Microcontroller Software Interface Standard (CMSIS)

- CMSIS is developed by ARM in close co-operation with various silicon and software vendors and provides a common approach to interface to peripherals, real-time operating systems, and middleware components. CMSIS applies to Cortex-M processor-based devices.

- CMSIS defines the basic requirements to achieve software re-usability and portability. MCU vendors may include additional functions for each peripheral to enrich the features of their software solution.

The area of standardization from CMSIS

- **Hardware Abstraction Layer (HAL) for Cortex-M processor registers** with standardized definitions for the SysTick, NVIC, System Control Block registers, MPU registers, and core access functions.

- **Standardized system exception names** to allow RTOS and middleware components to utilize system exceptions without having compatibility issues.

- **Standardized methods to organize header files** that makes it easy to learn new Cortex-M microcontroller products and improve software portability.

- **Common methods for system initialization** to be used by each MCU vendor. For example, the standardized ***SystemInit( )*** function, provided in each device driver library, is essential for configuring the clock.

- **Standardized intrinsic functions** that are normally used to produce instructions that cannot be generated by IEC/ISO C. By having standardized intrinsic functions, software re-usability and portability is greatly improved.

- **Standardized ways to determine the system clock frequency** through a software variable, *SystemFrequency*, defined in the device driver. Allows RTOS to setup the SysTick unit based on the system clock frequency.

CMSIS Organization

- **Core Peripheral Access Layer**

  provides name definitions, address definitions, and helper functions to access core registers and core peripherals.

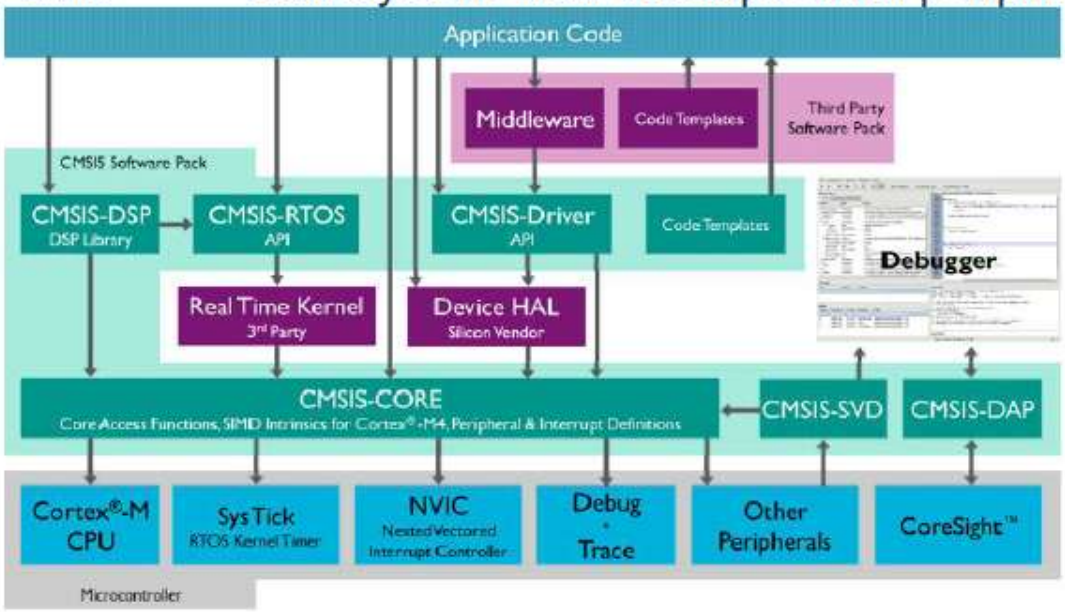
- **Device Peripheral Access Layer (MCU specific)**

  offers name definitions, address definitions, and driver code to access peripherals.

- **Access Functions for Peripherals (MCU specific and optional)** implements additional helper functions for peripherals.

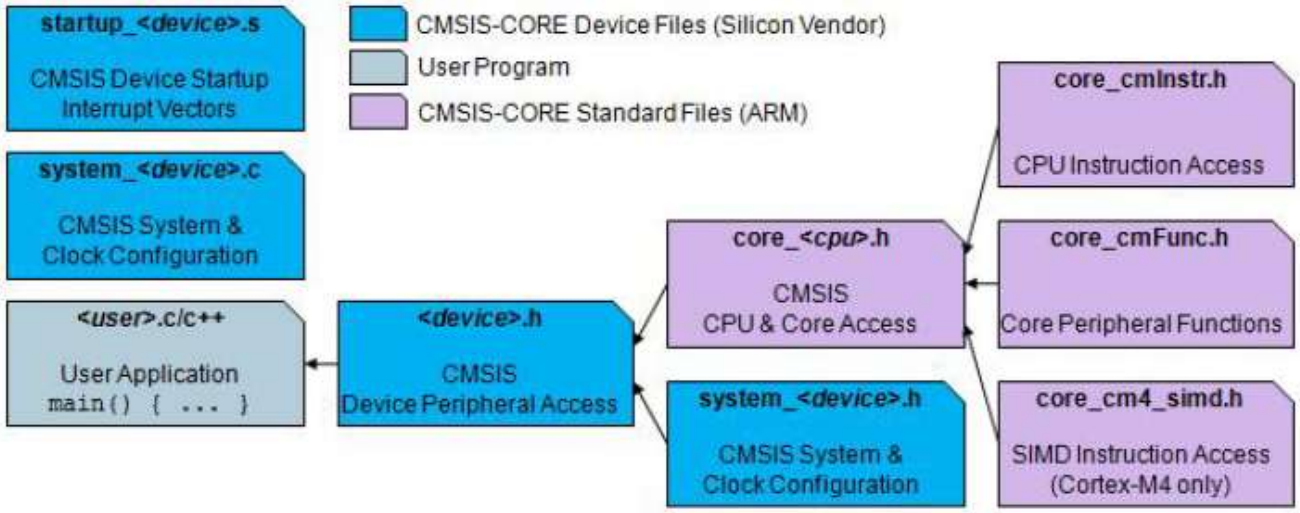
CMSIS Structure

- **There are seven main components to CMSIS:**
  - CMSIS-CORE API for Cortex-M processor and core peripherals
  - CMSIS-DRIVER API for generic peripheral drivers
  - CMSIS-DAP Debug Access Port firmware specification
  - CMSIS-DSP DSP Library with many function types for Cortex-M
  - CMSIS-PACK Device package (PDSC) for software development
  - CMSIS-RTOS API for RTOS integration
  - CMSIS-SVD XML system view description for peripherals



Using CMSIS-CORE

- **To use the CMSIS-CORE the following files are added to the embedded application:**
  - Startup File `startup_<device>.s` with reset handler and exception vectors
  - System Configuration Files `system_<device>.c` and `system_<device>.h` with general device configuration (clock setup)
  - Device Header File `<device.h>` gives access to processor core and all peripherals



- **Silicon vendors create these device-specific CMSIS-CORE files based on template files provided by ARM**

CMSIS Startup and Initialization

- **CMSIS-CORE provides the following files for an embedded application:**
  - **Startup File** `startup_<device>.s` with reset handler and exception vectors
  - **System Configuration Files** `system_<device>.c` **and** `system_<device.h>` with general device configuration (e.g., clocks, PLLs and bus setup)
  - **Device Header File** `<device.h>` gives access to processor core and all peripherals

- **For any existing device, ARM recommends including CMSIS-CORE startup and system initialization code from the device vendor**
  - If this is not available, startup and system initialization code will have to be created manually or by using an alternative, non-CMSIS compliant solution

- **When designing a new device, ARM recommends that the device vendor:**
  - Creates a CMSIS-SVD description to automatically generate CMSIS compliant device header files
    - Formalizes the programmer's view for device-specific peripherals
  - Adapts the system initialization and startup template files to their device

CMSIS-CORE: Startup File

- **The Startup File `startup_<device>.s` contains the following:**
  - Vector table
    - Exception vectors of the Cortex-M processor
    - Interrupt vectors that are device specific
    - Weak functions that implement default routines
  - Reset handler
    - calls `SystemInit` followed by `__main`
  - Stack and heap configuration
    - `__user_initial_stackheap` when using the standard C library
    - `__initial_sp`, `__heap_base` and `__heap_limit` when using microlib
    - discussed in more detail later …

- **The file exists for each supported toolchain and is the only toolchain specific CMSIS file**

CMSIS-CORE: Vector Table

```
; Vector Table Mapped to Address 0 at Reset
      AREA    RESET, DATA, READONLY, ALIGN=8
      EXPORT  __Vectors
      EXPORT  __Vectors_End
      EXPORT  __Vectors_Size

      __Vectors DCD     __initial_sp
                DCD     Reset_Handler
                DCD     NMI_Handler
                DCD     HardFault_Handler
                DCD     MemManage_Handler
                DCD     BusFault_Handler
                DCD     UsageFault_Handler
                DCD     0, 0, 0, 0
                DCD     SVC_Handler
                DCD     DebugMon_Handler
                DCD     0
                DCD     PendSV_Handler
                DCD     SysTick_Handler

; External Interrupts
; Add here the vectors for the device specific external interrupts handler
                DCD <DeviceInterrupt>_IRQHandler ; 0: Default

      __Vectors_End
      __Vectors_Size  EQU __Vectors_End - __Vectors
```

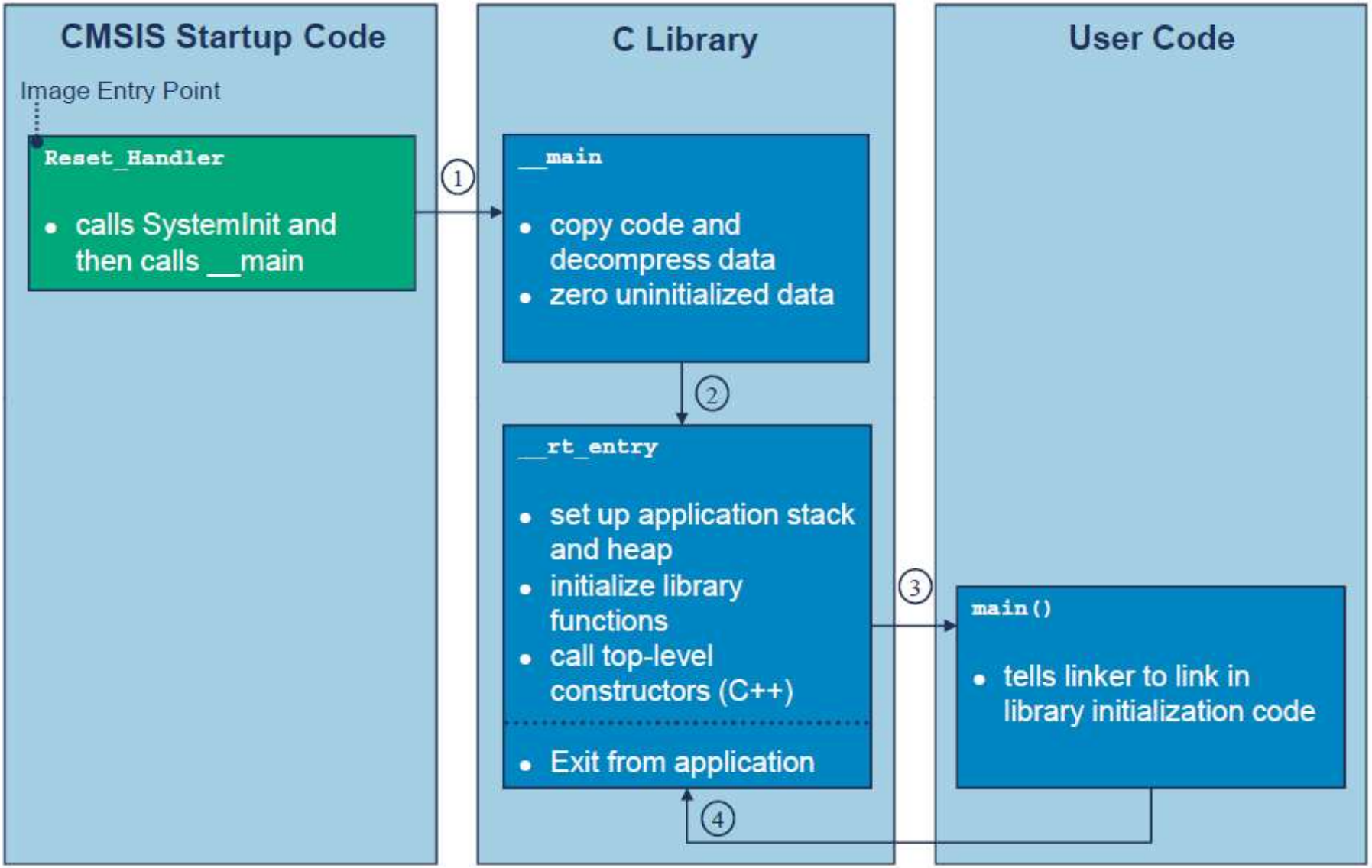
CMSIS-CORE: Exception Handlers

```
; Reset Handler
Reset_Handler           PROC
                        EXPORT Reset_Handler [WEAK]
                        IMPORT SystemInit
                        IMPORT __main
                        LDR R0, =SystemInit
                        BLX R0
                        LDR R0, =__main
                        BX R0
                        ENDP

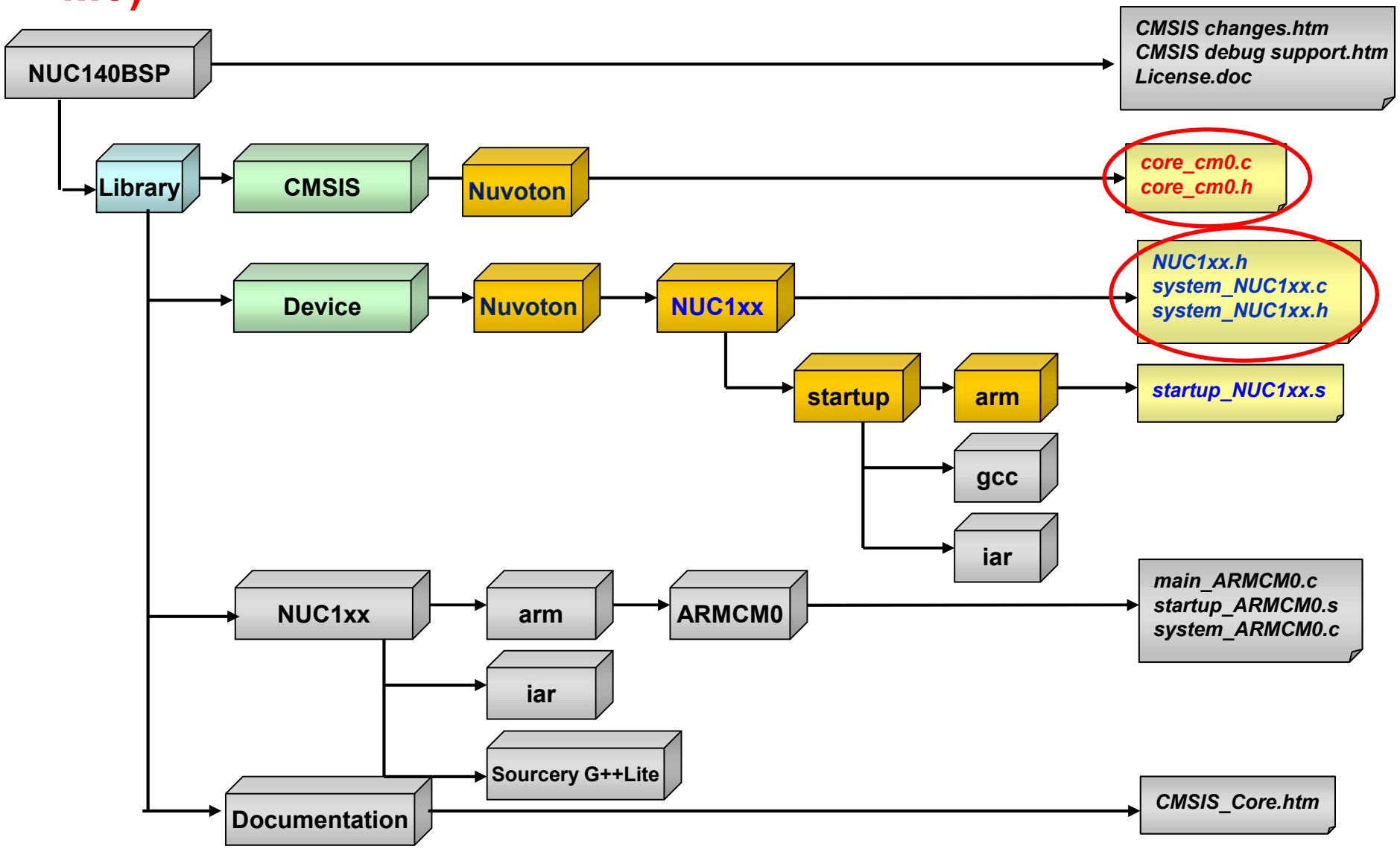
; Dummy Exception Handlers (infinite loops which can be modified)
NMI_Handler             PROC
                        EXPORT NMI_Handler [WEAK]
                        B .
                        ENDP

HardFault_Handler       PROC
                        EXPORT HardFault_Handler [WEAK]
                        B .
                        ENDP
```

Possible Initialization Sequence (1)



CMSIS directory structure (Nuvoton Cortex-M0)



CMSIS coding rule

- ANSI standard types defined in the ANSI C header file <stdint.h> are used

- For each exception/interrupt
  - An exception/interrupt handler rule
    - exception: the postfix _Handler
    - interrupts: the postfix _IRQHandler
  - A # define of the interrupt number with postfix _IRQn
  - A default exception/interrupt handler (**weak definition**) that contains an endless loop

What CMSIS Files Do You Need For NUC100 Series?

(NUVOTON Cortex-M0)

- NUC1xx.h (*device*.h)
  - Interrupt Number Definition
  - Device Peripheral Access Layer
    - Provides definitions for all device peripherals. It contains all data structures and the address mapping for the device specific peripherals
  - To define the peripherals for the actual device. It can use several other include files to define the peripherals of the actual devices
- startup_NUC1xx.s (startup_device.s)
  - Cortex-M0 startup code and the complete Interrupt Vector Table
- system_NUC1xx.c (system_device.c)

IC vendor offers 3 files

- core_cm0.h
- core_cm0.c

ARM offers 2 files

NUC100 Interrupt Number Definition: (NUVOTON Cortex-M0)

```
typedef enum IRQn
{
/****** Cortex-M0 Processor Exceptions Numbers ***********************************************/
  NonMaskableInt_IRQn       = -14,    /*!< 2 Non Maskable Interrupt                */
  HardFault_IRQn            = -13,    /*!< 3 Cortex-M0 Hard Fault Interrupt        */
  SVCall_IRQn               = -5,     /*!< 11 Cortex-M0 SV Call Interrupt          */
  PendSV_IRQn               = -2,     /*!< 14 Cortex-M0 Pend SV Interrupt          */
  SysTick_IRQn              = -1,     /*!< 15 Cortex-M0 System Tick Interrupt      */
/****** ARMIKMCU Swift specific Interrupt Numbers *********************************************/
  BOD_IRQn                  = 0,
  WDT_IRQn                  = 1,
  EINT0_IRQn                = 2,
  EINT1_IRQn                = 3,
  GPAB_IRQn                 = 4,
  GPCDE_IRQn                = 5,
  PWM0_IRQn                 = 6,
  PWM1_IRQn                 = 7,
  TMR0_IRQn                 = 8,
  TMR1_IRQn                 = 9,
  TMR2_IRQn                 = 10,
  TMR3_IRQn                 = 11,
  UART0_IRQn                = 12,
  UART1_IRQn                = 13,
  SPI0_IRQn                 = 14,
  SPI1_IRQn                 = 15,
  SPI2_IRQn                 = 16,
  SPI3_IRQn                 = 17,
  I2C0_IRQn                 = 18,
  I2C1_IRQn                 = 19,
  CAN0_IRQn                 = 20,
  CAN1_IRQn                 = 21,
  SD_IRQn                   = 22,
  USBD_IRQn                 = 23,
  PS2_IRQn                  = 24,
  ACMP_IRQn                 = 25,
  PDMA_IRQn                 = 26,
  I2S_IRQn                  = 27,
  PWRWU_IRQn                = 28,
  ADC_IRQn                  = 29,
  DAC_IRQn                  = 30,
  RTC_IRQn                  = 31
} IRQn_Type;
```

The interrupt number is defined in the “NUC1xx.h” file

startup_NUC1xx.s [1] (NUVOTON Cortex-M0)

```
_Vectors  DCD    __initial_sp              ; Top of Stack
          DCD    Reset_Handler             ; Reset Handler
          DCD    NMI_Handler               ; NMI Handler
          DCD    HardFault_Handler         ; Hard Fault Handler
          DCD    0                         ; Reserved
          DCD    0                         ; Reserved
          DCD    0                         ; Reserved
          DCD    0                         ; Reserved
          DCD    0                         ; Reserved
          DCD    0                         ; Reserved
          DCD    0                         ; Reserved
          DCD    SVC_Handler               ; SVCall Handler
          DCD    0                         ; Reserved
          DCD    0                         ; Reserved
          DCD    PendSV_Handler            ; PendSV Handler
          DCD    SysTick_Handler           ; SysTick Handler
```

These exception names are fixed and define the start of the vector table for a Cortex-M0

startup_NUC1xx.s [2]

(NUVOTON Cortex-M0)

```
; External Interrupts
                DCD     BOD_IRQHandler
                DCD     WDT_IRQHandler
                DCD     EINT0_IRQHandler
                DCD     EINT1_IRQHandler
                DCD     GPAB_IRQHandler
                DCD     GPCDE_IRQHandler
                DCD     PWMA_IRQHandler
                DCD     PWMB_IRQHandler
                DCD     TMR0_IRQHandler
                DCD     TMR1_IRQHandler
                DCD     TMR2_IRQHandler
                DCD     TMR3_IRQHandler
                DCD     UART02_IRQHandler
                DCD     UART1_IRQHandler
                DCD     SPI0_IRQHandler
                DCD     SPI1_IRQHandler
                DCD     SPI2_IRQHandler
                DCD     SPI3_IRQHandler
                DCD     I2C0_IRQHandler
                DCD     I2C1_IRQHandler
                DCD     CAN0_IRQHandler
                DCD     Default_Handler
                DCD     Default_Handler
                DCD     USBD_IRQHandler
                DCD     PS2_IRQHandler
                DCD     ACMP_IRQHandler
                DCD     PDMA_IRQHandler
                DCD     I2S_IRQHandler
                DCD     PWRWU_IRQHandler
                DCD     ADC_IRQHandler
                DCD     Default_Handler
                DCD     RTC_IRQHandler
```

These interrupt names are fixed and define the start of the vector table for a Cortex-M0

Memory Map defined in NUC1xx.h

(NUVOTON Cortex-M0)

```
/* Peripheral and SRAM base address */
#define FLASH_BASE        ((    uint32_t)0x00000000)
#define SRAM_BASE         ((    uint32_t)0x20000000)
#define AHB_BASE          ((    uint32_t)0x50000000)
#define APB1_BASE         ((    uint32_t)0x40000000)
#define APB2_BASE         ((    uint32_t)0x40100000)

/* Peripheral memory map */
#define GPIO_BASE         (AHB_BASE      + 0x4000)

#define GPIOA_BASE        (GPIO_BASE           )
#define GPIOB_BASE        (GPIO_BASE     + 0x0040)
#define GPIOC_BASE        (GPIO_BASE     + 0x0080)
#define GPIOD_BASE        (GPIO_BASE     + 0x00C0)
#define GPIOE_BASE        (GPIO_BASE     + 0x0100)
#define GPIO_DBNCECON_BASE (GPIO_BASE    + 0x0180)

#define UART0_BASE        (APB1_BASE     + 0x50000)
#define UART1_BASE        (APB2_BASE     + 0x50000)
#define UART2_BASE        (APB2_BASE     + 0x54000)
```

```
#define TIMER0_BASE       (APB1_BASE     + 0x10000)
#define TIMER1_BASE       (APB1_BASE     + 0x10020)
#define TIMER2_BASE       (APB2_BASE     + 0x10000)
#define TIMER3_BASE       (APB2_BASE     + 0x10020)
#define WDT_BASE          (APB1_BASE     + 0x4000)
#define SPI0_BASE         (APB1_BASE     + 0x30000)
#define SPI1_BASE         (APB1_BASE     + 0x34000)
#define SPI2_BASE         (APB2_BASE     + 0x30000)
#define SPI3_BASE         (APB2_BASE     + 0x34000)
#define I2C0_BASE         (APB1_BASE     + 0x20000)
#define I2C1_BASE         (APB2_BASE     + 0x20000)
#define RTC_BASE          (APB1_BASE     + 0x08000)
#define ADC_BASE          (APB1_BASE     + 0xE0000)
#define ACMP_BASE         (APB1_BASE     + 0xD0000)
#define SYSCLK_BASE       (AHB_BASE      + 0x00200)
#define GCR_BASE          (AHB_BASE      + 0x00000)
#define INT_BASE          (AHB_BASE      + 0x00300)
#define FMC_BASE          (AHB_BASE      + 0x0C000)
#define PS2_BASE          (APB2_BASE     + 0x00000)
#define CAN0_BASE         (APB2_BASE     + 0x80000)
#define CAN1_BASE         (APB2_BASE     + 0x84000)
#define C_CAN_BASE        (APB2_BASE     + 0x88000)
```

Peripheral Name define its Base Address
(NUVOTON Cortex-M0)

```
#define GPIOA               ((GPIO_T *) GPIOA_BASE)
#define GPIOB               ((GPIO_T *) GPIOB_BASE)
#define GPIOC               ((GPIO_T *) GPIOC_BASE)
#define GPIOD               ((GPIO_T *) GPIOD_BASE)
#define GPIOE               ((GPIO_T *) GPIOE_BASE)
#define GPIO_DBNCECON       ((GPIO_DBNCECON_T *)
GPIO_DBNCECON_BASE)
#define GPIO_BIT_ADDR_BASE  (GPIO_BASE + 0x200)

#define UART0               ((UART_T *) UART0_BASE)
#define UART1               ((UART_T *) UART1_BASE)
#define UART2               ((UART_T *) UART2_BASE)
#define TIMER0              ((TIMER_T *) TIMER0_BASE)
#define TIMER1              ((TIMER_T *) TIMER1_BASE)
#define TIMER2              ((TIMER_T *) TIMER2_BASE)
#define TIMER3              ((TIMER_T *) TIMER3_BASE)
#define WDT                 ((WDT_T *) WDT_BASE)
#define SPI0                ((SPI_T *) SPI0_BASE)
#define SPI1                ((SPI_T *) SPI1_BASE)
#define SPI2                ((SPI_T *) SPI2_BASE)
#define SPI3                ((SPI_T *) SPI3_BASE)
```

```
#define I2C0                ((I2C_T *) I2C0_BASE)
#define I2C1                ((I2C_T *) I2C1_BASE)
#define I2S                 ((I2S_T *) I2S_BASE)
#define RTC                 ((RTC_T *) RTC_BASE)
#define ADC                 ((ADC_T *) ADC_BASE)
#define ACMP                ((ACMP_T *) ACMP_BASE)
#define SYSCLK              ((SYSCLK_T *) SYSCLK_BASE)
#define SYS                 ((GCR_T *) GCR_BASE)
#define SYSINT              ((GCR_INT_T *) INT_BASE)
#define FMC                 ((FMC_T *) FMC_BASE)
#define PS2                 ((PS2_T *) PS2_BASE)
#define CAN                 ((CAN_T *) CAN0_BASE)
#define USBD                ((USBD_T *) USBD_BASE))
```